CV802

Assignment 3: Novel-view Synthesis

Anas Khan

Problem Statement:



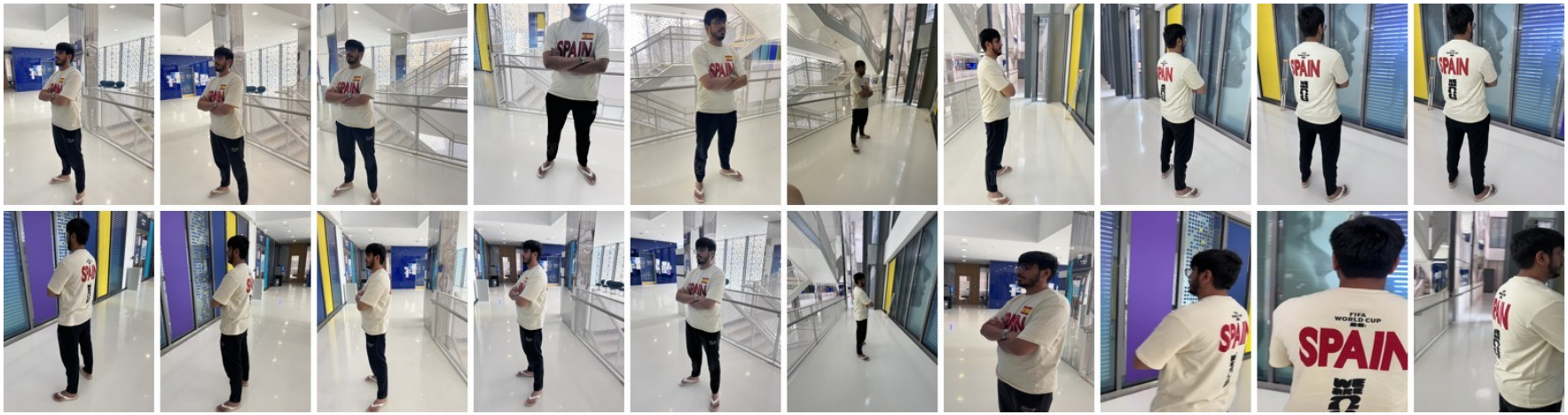


125 photos + their cameras (Assignment 1)

pictures from new viewpoints

Output: a video along a preset camera path (.mp4) and drag-to-view inside the starter GUI

Inputs



- The same 125 photos as Assignments 1 and 2 (4 iPhone lenses + video frames)
- Cameras from Assignment 1 (VGGSfM), undistorted to pinhole
- Person masks → person on a white background; resized to 1000 px
- 109 photos for training, 16 held out for scoring (every 8th, the 3DGS rule)
- No point cloud: Gaussians start from random points

Methods

Method	Venue	Year	Scene stored as
NeRF	**ECCV**	**2020**	**one neural network (8 layers × 256)**
TensoRF	**ECCV**	**2022**	**a grid split into planes and lines**
3D Gaussian Splatting (3DGS)	**ACM TOG / SIGGRAPH**	**2023**	**~150k coloured 3D blobs**
Mip-Splatting	**CVPR**	**2024**	**3DGS + two anti-aliasing filters**

Official code (NeRF: nerf-pytorch, the standard PyTorch version of the authors' TensorFlow code), same photos and cameras.
TensoRF, 3DGS, Mip-Splatting: 30,000 steps. NeRF: 60 minutes of training (the paper trains for 200,000+ steps).

TensoRF: how it works

- Space is a 3D grid of features: density (how solid) and colour
- A full 300×300×300 grid is huge, so it is stored as planes × lines:
  - feature(x, y, z) = Σ plane_XY(x, y) · line_Z(z) + plane_XZ · line_Y + plane_YZ · line_X
- A pixel = shoot a ray, sample points along it, add up their colours:
  - $C = \Sigma\ T_i \cdot (1 - e^{(-\sigma_i \delta_i)}) \cdot c_i$, T_i = how much light still gets through
- Small network turns the feature + viewing direction into a colour

3D Gaussian Splatting: how it works

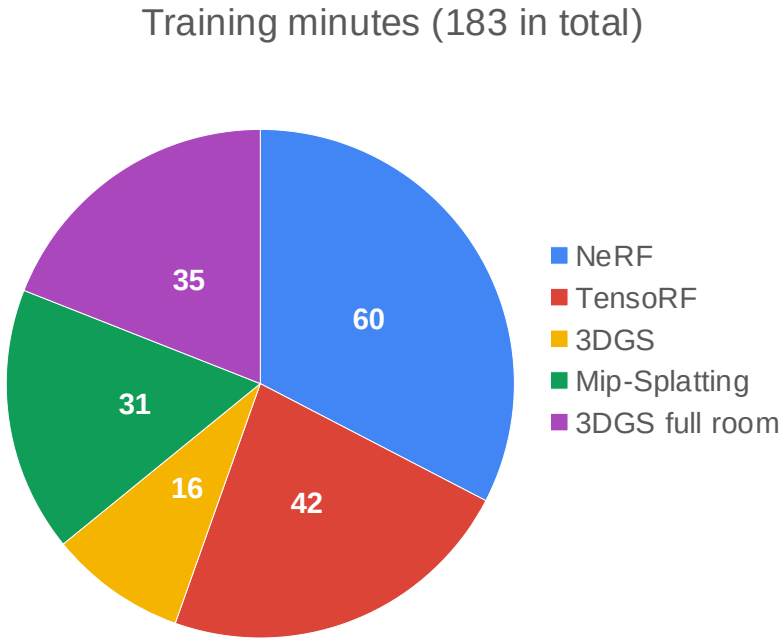
- The scene is ~150k 3D Gaussians (soft ellipsoids): position, size, rotation, opacity, colour
- Start from 100k random points (input = photos + cameras only); add, split, delete blobs while training
- To draw: project each blob to the screen (a 2D ellipse), sort front to back, blend:
  - $\Sigma' = J\,W\,\Sigma\,W^T\,J^T$ (3D shape → 2D shape on screen)
  - $C = \Sigma\ c_i\,\alpha_i\,\Pi_{j(j<i)}\,(1-\alpha_j)$
- No ray marching → real time (hundreds of frames per second)

Mip-Splatting: how it works

- 3DGS, plus two filters so a blob is never sharper than the camera can see
- 3D filter: each blob gets a minimum size, set by the closest training photo
  - $\Sigma \leftarrow \Sigma + s^2 \cdot I$ (no blobs thinner than one pixel of any photo)
- 2D filter: on screen, each blob is blurred to at least one pixel
- Why it matters: zoom out and 3DGS blobs become too thin → flicker and jagged edges
- Here also better at the training size, and far better when the camera moves away

Experiments

- 5 trainings: NeRF, TensoRF, 3DGS, Mip-Splatting (person), 3DGS full room (extra)
- Scores on the 16 held-out photos: PSNR, SSIM, LPIPS
- Zoom test: render 3DGS and Mip-Splatting at ¼ size and 1.6× size
- Ablation: Gaussians from random points vs from the Assignment 1 point cloud
- Result video: same 8 s orbit for every method
- Check: our GUI renderer vs the official renderer (same pixels?)



One RTX 5000 Ada (32 GB), trainings run side by side.

Results on held-out photos



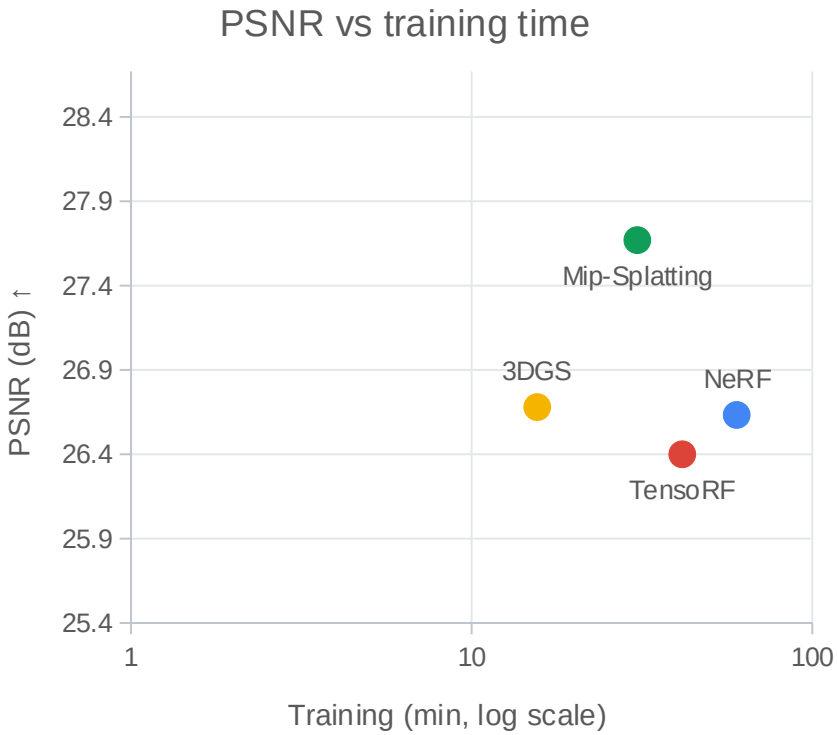
Comparison

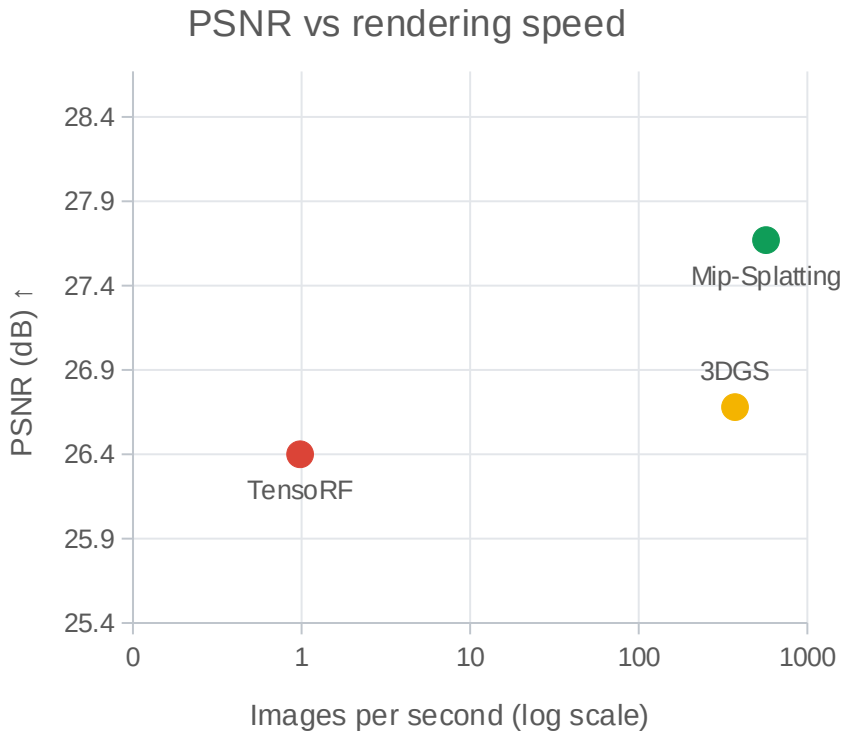
best second best

Method	PSNR ↑	SSIM ↑	LPIPS ↓	Training ↓	FPS ↑	Primitives	Size (MB)
NeRF	**26.59**	**0.931**	**0.132**	**60 min**	**0.04**	**1.2M**	**14**
TensoRF	**26.35**	**0.938**	**0.130**	**42 min**	**0.98**	**17.4M**	**67**
3DGS	**26.63**	**0.951**	**0.072**	**16 min**	**373**	**149k**	**35**
Mip-Splatting	**27.62**	**0.958**	**0.067**	**31 min**	**569**	**197k**	**47**

16 held-out photos, 750×1000. PSNR in dB. FPS = images per second on the GPU. Primitives: Gaussians (3DGS, Mip) or numbers in the network / grid (NeRF, TensoRF). Gaussians start from random points.

Quality vs time and speed





Higher is better; left chart: further left is faster to train, right chart: further right draws faster.

Zoom test: 3DGS vs Mip-Splatting



Rendered at	3DGS	Mip
¼ size	23.71	28.63
1× (training)	26.63	27.62
1.6×	26.29	27.37

Left: rendered at ¼ size, enlarged ×6 (pixels kept square). Table: PSNR (dB), 16 held-out photos; ¼ = photo averaged down, 1.6× = original 1600 px photos.

Zoomed out, 3DGS blobs get thinner than a pixel and the image breaks up; Mip-Splatting stays clean.

Ablation: random start vs point cloud

Start	3DGS PSNR ↑	3DGS LPIPS ↓	Mip PSNR ↑	Mip LPIPS ↓	Room PSNR ↑
100k random points (main)	**26.63**	**0.072**	**27.62**	**0.067**	**17.96**
Assignment 1 point cloud	**27.53**	**0.064**	**27.53**	**0.064**	**19.71**

- Main results use only the photos and their cameras: every Gaussian starts at a random position
- The point cloud of Assignment 1 is only a better starting guess; training does the rest
- Same photos, cameras, 30,000 steps and scoring in both rows

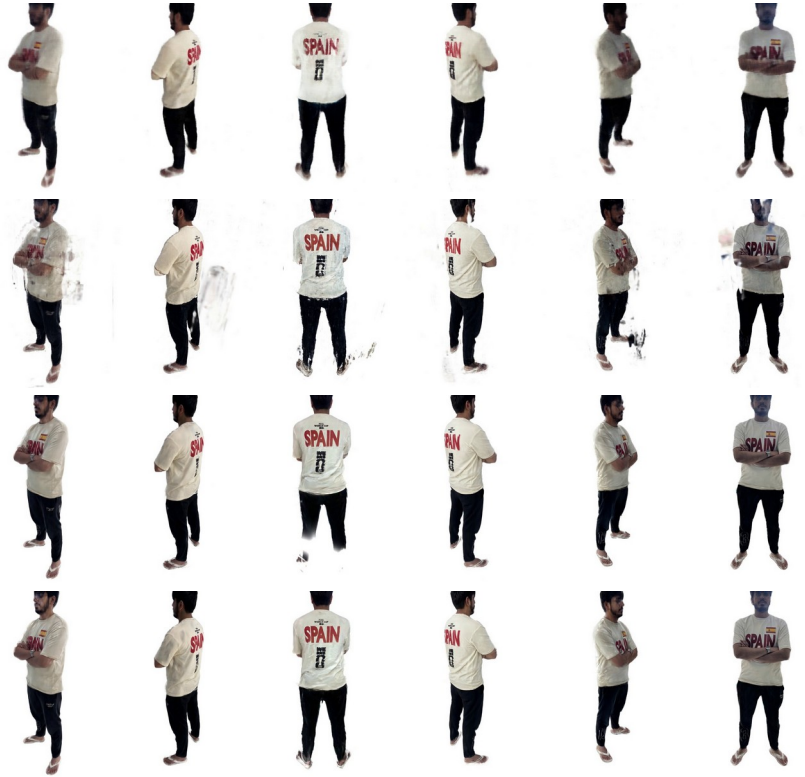
Result videos (preset 8 s orbit)

NeRF

TensoRF

3DGS

Mip-Splatting



Frames 0, 40, 80, 120, 160, 200 of 240. The full videos are in results/videos/.

Extra: 3DGS with the room



- Same photos without the person masks: 17.96 dB PSNR on the held-out photos
- 0.91M Gaussians (person alone: ~150k), trained in 35 min
- The room is only seen from where the photos were taken, so views far from them break up

GUI: inside the course starter code



What the GUI's 3D view draws, from one held-out photo's camera (offscreen render of the same code)

- One app for Assignments 1, 2 and 3 (the course starter)
- File > Open novel-view results; method menu
- Drag / pan / zoom → the novel view follows
- Held-out photo vs render, one click

- Keyframes or preset orbit → Export MP4
- Assignment 2 in the app: Extract mesh (our Poisson) + rasterised
- Runs on a laptop: our PyTorch renderers
  - (CUDA, Apple GPU or CPU)

GUI renderer = official renderer

Model	Same pixels? (PSNR vs official)	Official FPS	Ours FPS (CUDA)
3DGS	**105 dB**	**373**	**4.6**
Mip-Splatting	**103 dB**	**569**	**5.1**

- Above ~50 dB two images look identical; >100 dB = differences of rounding only
- Ours follows the CUDA code step by step, in plain PyTorch, so it also runs without CUDA
- Slower than CUDA, so the GUI draws a small preview while dragging, full size when the mouse stops

Findings

- Best quality on held-out photos: Mip-Splatting (27.62 dB)
- Fastest to train: 3DGS (16 min); Gaussians draw in real time
- Mip-Splatting is best at every size; zoomed out by far (+4.9 dB)
- No point cloud needed: from random points Mip-Splatting is even slightly better; 3DGS loses ~0.9 dB
- Clean masks matter: the white background makes the person easy to learn
- NeRF: same rendering idea as TensoRF, but one big network → slowest to train and to draw

References

[1] B. Mildenhall et al. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. ECCV 2020.
[2] A. Chen, Z. Xu, A. Geiger, J. Yu, H. Su. TensoRF: Tensorial Radiance Fields. ECCV 2022.
[3] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. ACM TOG 42(4), 2023.
[4] Z. Yu, A. Chen, B. Huang, T. Sattler, A. Geiger. Mip-Splatting: Alias-free 3D Gaussian Splatting. CVPR 2024.
[5] M. Zwicker, H. Pfister, J. van Baar, M. Gross. EWA Volume Splatting. IEEE Visualization 2001.
[6] J. Wang, N. Karaev, C. Rupprecht, D. Novotny. VGGSfM: Visual Geometry Grounded Deep Structure from Motion. CVPR 2024.
[7] Z. Wang, A. C. Bovik, H. R. Sheikh, E. P. Simoncelli. Image Quality Assessment: From Error Visibility to Structural Similarity. IEEE TIP 13(4), 2004.
[8] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, O. Wang. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric. CVPR 2018.
[9] Q.-Y. Zhou, J. Park, V. Koltun. Open3D: A Modern Library for 3D Data Processing. arXiv:1801.09847, 2018.
[10] M. Tancik et al. Nerfstudio: A Modular Framework for Neural Radiance Field Development. SIGGRAPH 2023.